宁波大学

本科实验报告

姓名：	梁航锋
学院：	信息科学与工程学院
系：	计算机科学与技术系
专业：	计算机科学与技术
学号：	216001912
指导教师：	董理

2024 年 3 月 14 日

宁波大学实验报告

课程名称：机器学习 实验类型：验证性

实验项目名称：实验四：神经网络与卷积神经网络

学生姓名：梁航锋 专业：计算机科学与技术 学号：216001912

同组学生姓名：

指导老师：董理

实验地点：计算机组成实验室 201 实验日期：2024 年 4 月 24 日

一、实验目的和要求

1. 学习掌握神经网络的基本原理，包括其组成结构、训练方式，理解基于梯度下降法的神经网络参数调整；

2. 体会传统神经网络模型在图像分类任务中的参数量，理解卷积神经网络的诞生意义与优化原理，并学习使用卷积神经网络进行图像分类的方法；

3. 学习使用 MindSpore 框架进行模型的定义与训练；

4. 实现 LeNet 的代码编写，并基于 MNIST 手写数字进行网络训练，并掌握一定的调参方法，使模型有更好的性能表现。

二、实验内容和原理

1. 导入 MNIST 手写数字数据集，并适当可视化以理解数据含义；

2. 使用 MindSpore 框架实现 LeNet，并进行模型的训练；

3. 保存模型的训练结果，并将训练好的模型参数加载至网络中，选取一个 batch 的数据进行数据可视化与预测；

4. 尝试进行模型参数调整，例如学习率、epochs、batchsize、权重初始化等，以优化模型的泛化性能。

5. 完成实验报告的攥写，并及时提交实验报告。

三、实验环境

操作系统：

Windows 11 Version 22H2

集成开发环境：

1. Anaconda3 with Jupyter Notebook
2. PyCharm 2022.2.2 (Professional Edition)

相关 Python 库：

1. python3.9
2. numpy
3. matplotlib（3.8.3）
4. sklearn（scikit-learn1.4.1）
5. MindSpore2.2.13 等

四、操作方法与实验步骤

1. 数据集加载

本次实验使用卷积神经网络（LeNet）进行多分类问题（手写数字识别）的求解，模型的训练与验证基于 MNIST 手写数字数据集进行，该数据集容易在互联网上获取，本次实验采用实验指导给出的二进制形式的数据集。

为了方便数据集的遍历与管理，实验代码实现了数据集类，其中类内部进行了数据集的加载、预处理、访问接口的代码编写，并实现了功能的封装，在调用处只需传入数据集路径、是否打乱、批次大小等信息即可完成数据集的加载，以供后续基于 MindSpore 框架的模型训练。

同时，为了方便对数据集信息进行查看，实验代码实现了可视化接口，该接口传入图像所对应的矩阵数组，输出对应的图像图片，以便于后续的模型测试分析。

2. 模型的实现

对于神经网络模型，由于网络参数量与神经元数目巨大，手动实现网络结构与训练逻辑相对较为困难，为简化模型的实现过程，许多深度学习框架提供了神经网络的定义逻辑。在 MindSpore 中，继承自 nn.Cell 的类可以通过重写 construct 方法来定义网络的正向传播逻辑，在框架内部将自动根据正向传播逻辑的实现代码来计算梯度的反向传播路径。

对于手写识别任务，经典的卷积神经网络具有良好表现，本次实验使用

LeNet 进行手写数字识别。模型的逻辑代码采用 MindSpore 框架实现。

对于 LeNet，使用卷积层和池化层进行数据的特征提取，采用全连接层实现从特征到类别的关系映射，首先待识别图像将经过一个卷积层，后经过一层激活函数（ReLU）进行激活，再通过池化层减少特征尺度，重复以上操作 2 遍，得到高维的抽象特征，再将特征按行展开排列，使用 3 次全连接+激活的方式得到输出层，输出层的每一个神经元代表分类到各个类别的概率。

具体的是实现代码见报告第五节。

3. 检查点设置

为了保存模型的训练结果，需要进行检查点设置，对于 MindSpore，需要提前声明网络模型在训练过程中是否需要保存中间过程和结果，通过 CheckpointConfig 和 ModelCheckpoint 类进行检查点的信息配置，如模型的保存路径、检查点保存时刻、文件命名等信息。

4. 模型训练参数的配置

模型的训练需要确定一系列参数，包括训练轮次，学习率，数据批量大小，损失函数等。

对于损失函数，多分类问题中常用交叉熵损失函数，其表达式如下：

$$Loss = -\sum_{i} y_i \log(f(x_i))$$

其中 yi 表示真实情况下是否为该类，即只取预测为对应类别的概率，f 为模型的输出概率。

学习率方面，初始不宜设置太大，本次实验选用 0.01。对于训练过程，深度学习框架通常提供对应的优化器以加速训练过程，例如 MindSpore 中的 Momentum 优化器，需要传入模型参数（使用 trainable_params()方法访问）以及学习率。

以上所有的相关训练参数最终通过 mindspore.Model 类进行包装，返回一个可训练的模型对象，调用该对象的 train 函数并传入数据集、epochs、检查点信息即可进行训练。

5. 模型训练与验证

调用 train 函数即可实现模型的训练，同步会将训练过程的相关信息进行

打印，使用 eval 函数测试模型在测试集上的表现。完成训练后能够得到 checkpoint，使用 load_checkpoint 可以载入对应检查点中的模型参数，使用 load_param_into_net 可以将参数加载到指定的网络实例中，测试一个批次的数据，进行可视化并查看模型的输出情况。

6. 模型参数调整

对模型的参数进行调整，以得到更优的测试集准确率。

五、实验数据记录和处理

该部分将按照代码逻辑与以上实验步骤进行代码的划分（划分为 notebook 的代码块），并记录。

5.1 相关库导入

```
# 相关库导入
from mindvision.dataset import Mnist
import mindspore.nn as nn
import mindspore as ms
from mindvision.engine.callback import LossMonitor
import numpy as npimport matplotlib.pyplot as plt
```

5.2 数据集的导入

```
# 数据集准备
download_train = Mnist(
    path="./mnist",
    split="train",
    batch_size=32,
    repeat_num=1,
    shuffle=True,
    resize=32,
    download=True)
download_eval = Mnist(
    path="./mnist",
    split="test",
    batch_size=32,
    resize=32,
    download=True)
dataset_train = download_train.run()
dataset_eval = download_eval.run()
```

5.3 LeNet 的实现

```
# 随机种子
seed_value = 2024
model = RandomForestClassifier(random_state=seed_value)
# 模型训练
model.fit(x_train, y_train)
# 在训练集、测试集上进行预测
y_train_pred = model.predict(x_train)
y_test_pred = model.predict(x_test)
# 模型定义
class LeNet5(nn.Cell):
    def __init__(self, num_class=10, num_channel=1):
        super(LeNet5, self).__init__()
        # 卷积层，输入通道数为num_channel，输出通道数为6，卷积核大小5*5
        self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
        # 卷积层，输入的通道数为6，输出的通道数为16，卷积核大小为5*5
        self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
        # 全连接层，输入个数为16*5*5，输出个数为120
        self.fc1 = nn.Dense(16 * 5 * 5, 120)
        # 全连接层，输入个数为120，输出个数为84
        self.fc2 = nn.Dense(120, 84)
        # 全连接层，输入个数为84，分类的个数为num_class
        self.fc3 = nn.Dense(84, num_class)
        # ReLU 激活函数
        self.relu = nn.ReLU()
        # 池化层
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        # 多维数组展平为一维数组
        self.flatten = nn.Flatten()

    def construct(self, x):
        # 使用定义好的运算构建前向网络
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
```

```
        return x
```

5.4 训练参数配置与模型训练

```
# 模型实例化
net=LeNet5(num_class=10)
# 损失函数：交叉熵损失
loss=nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
# 优化器
optimizer=nn.Momentum(net.trainable_params(),
                      learning_rate=0.01, momentum=0.9)
# MindSpore 需要提前声明网络模型在训练过程中是否需要保存中间过程和结果
# 设置模型保存参数
config_ck = ms.CheckpointConfig(save_checkpoint_steps=1875, keep_chec
kpoint_max=10)
# 应用模型保存参数
checkpoint = ms.ModelCheckpoint(prefix="lenet", directory="./lenet",
config=config_ck)
# 模型训练初始化
model = ms.Model(network=net,
                 loss_fn=loss,
                 optimizer=optimizer,
                 metrics={'accuracy'})
# 训练网络模型
model.train(10, dataset_train,
                callbacks=[checkpoint, LossMonitor(0.01, 1875)])
```

5.5 模型测试

```
# 在测试集上进行模型的表现测试
acc = model.eval(dataset_eval)
print("{}".format(acc))
# 加载模型的参数
param_dict = ms.load_checkpoint("./lenet/lenet-1_1875.ckpt")
# 加载参数到网络中
ms.load_param_into_net(net, param_dict)
# 测试模型并可视化结果
mnist = Mnist("./mnist", split="train", batch_size=9, resize=32)
dataset_infer = mnist.run()
ds_test = dataset_infer.create_dict_iterator()
data = next(ds_test)
images = data["image"].asnumpy()
labels = data["label"].asnumpy()
plt.figure()
for i in range(1, 10):
    plt.subplot(3, 3, i)
```

```
    plt.imshow(images[i-1][0], interpolation="None", cmap="gray")
plt.show()
# 使用函数 model.predict 预测 image 对应分类
output = model.predict(ms.Tensor(data['image']))
predicted = np.argmax(output.asnumpy(), axis=1)
# 输出预测分类与实际分类
print(f'Predicted: "{predicted}", Actual: "{labels}"')
```

六、实验结果与分析

该部分将对实验代码产出的相关结果与图像进行记录与分析。

1. 数据可视化

给定的数据为 28*28 的图像，使用 matplotlib 库进行绘图，选取 9 个样本数据，绘图结果如图 6.1 所示。可以看到数据集以 28*28 的图像形式展现手写数字，也是神经网络的输入。

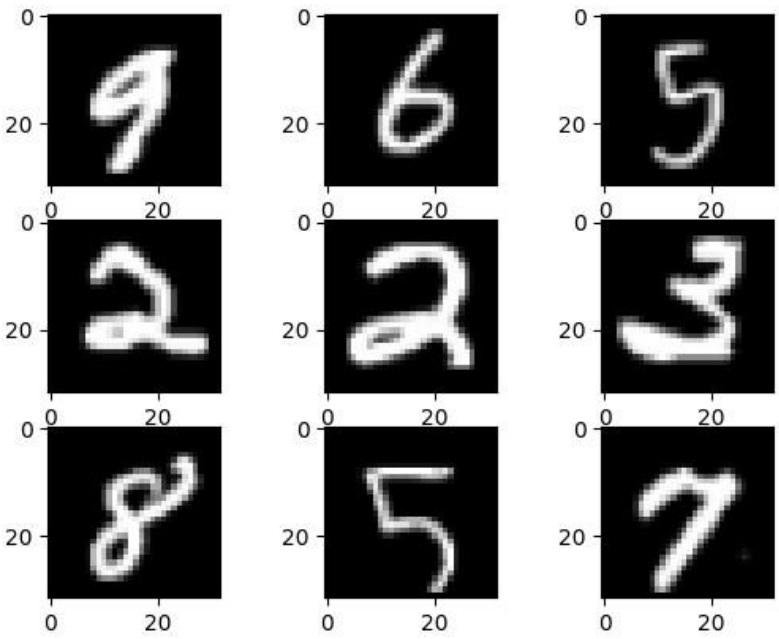


图 6.1 MNIST 手写数字数据可视化

2. 数据集的导入

运行代码，实现数据集的导入，使用 len 查看数据集的规模，如图 6.2 所示。其中训练集包含 1875 个样本，测试集包含 312 个样本。

```
len(dataset_train), len(dataset_eval)
```

```
(1875, 312)
```

图 6.2 数据集大小

3. 模型训练过程

运行框架的模型训练函数，模型开始训练，并打印了训练过程，如图 6.3 所示。可以看到每一个 epoch 遍历训练完一遍训练集，损失逐渐下降，从初始的 0.197 到 0.013

```
# 训练网络模型
model.train(10, dataset_train, callbacks=[checkpoint, LossMonitor(0.01, 1875)])
```

```
Epoch:[  0/ 10], step:[ 1875/ 1875], loss:[0.006/0.197], time:15.060 ms, lr:0.01000
Epoch time: 389551.466 ms, per step time: 207.761 ms, avg loss: 0.197
Epoch:[  1/ 10], step:[ 1875/ 1875], loss:[0.015/0.059], time:9.241 ms, lr:0.01000
Epoch time: 8889.325 ms, per step time: 4.741 ms, avg loss: 0.059
Epoch:[  2/ 10], step:[ 1875/ 1875], loss:[0.017/0.041], time:10.002 ms, lr:0.01000
Epoch time: 8934.486 ms, per step time: 4.765 ms, avg loss: 0.041
Epoch:[  3/ 10], step:[ 1875/ 1875], loss:[0.025/0.032], time:9.002 ms, lr:0.01000
Epoch time: 8417.905 ms, per step time: 4.490 ms, avg loss: 0.032
Epoch:[  4/ 10], step:[ 1875/ 1875], loss:[0.000/0.026], time:9.105 ms, lr:0.01000
Epoch time: 8326.954 ms, per step time: 4.441 ms, avg loss: 0.026
Epoch:[  5/ 10], step:[ 1875/ 1875], loss:[0.002/0.022], time:9.003 ms, lr:0.01000
Epoch time: 8996.591 ms, per step time: 4.798 ms, avg loss: 0.022
Epoch:[  6/ 10], step:[ 1875/ 1875], loss:[0.013/0.020], time:10.002 ms, lr:0.01000
Epoch time: 8864.220 ms, per step time: 4.728 ms, avg loss: 0.020
Epoch:[  7/ 10], step:[ 1875/ 1875], loss:[0.000/0.020], time:9.391 ms, lr:0.01000
Epoch time: 9154.357 ms, per step time: 4.882 ms, avg loss: 0.020
Epoch:[  8/ 10], step:[ 1875/ 1875], loss:[0.001/0.015], time:9.129 ms, lr:0.01000
Epoch time: 8438.488 ms, per step time: 4.501 ms, avg loss: 0.015
Epoch:[  9/ 10], step:[ 1875/ 1875], loss:[0.000/0.013], time:9.157 ms, lr:0.01000
Epoch time: 8230.529 ms, per step time: 4.390 ms, avg loss: 0.013
```

图 6.3 模型训练过程

4. 模型测试

使用 eval 函数进行模型测试，使用测试集进行模型的推理，该函数直接返回最终的准确率，结果如图 6.4 所示。可以看到模型的准确率达到了 98.86%，已有较好的模型表现。

```
# 在测试集上进行模型的表现测试
acc = model.eval(dataset_eval)
print("{}".format(acc))
```

```
{'accuracy': 0.9885817307692307}
```

图 6.4 模型测试

由于前面训练过程中已保存了一系列 checkpoints，可以直接加载模型进行进一步的测试。加载模型后，随机选定测试集中的九个样本，显示对应的图片、标签，并输出模型的预测结果，如图 6.5 所示。

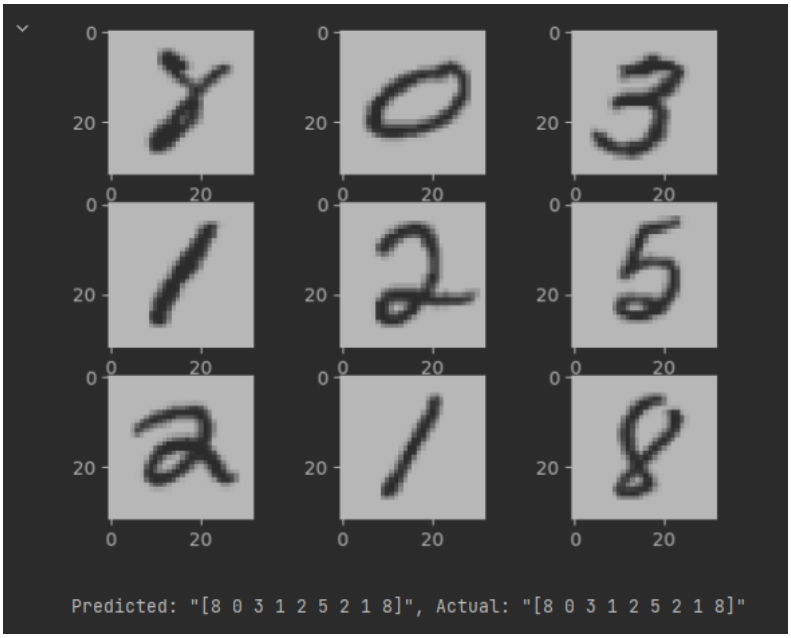


图 6.5 部分样本的测试

可以看到，模型的预测结果与真实情况吻合，能够较好的对手写数字图片进行分类，具有良好性能。

通过调整超参数，将训练批次增加，并适当减小学习率以求达到更精确的最优解，结果如图 6.6 所示。准确率相对图 6.6 略有提升，达到 98.97%的准确率，但提升不明显，事实上模型的精确度已经较高了，继续训练可能会降低模型的泛化性能。

```
acc = model.eval(dataset_eval)
print("{}".format(acc))
```

```
{'accuracy': 0.9896834935897436}
```

图 6.6 调参后的模型性能

七、讨论、心得

本次实验基于 MindSpore 框架进行了手写数字识别任务的实现，并在 MNIST 数据集上进行模型的训练与测试，最终模型表现出良好性能，在测试数据计算上表现出 98%的准确率。

具体地，本次实验采用的神经网络模型为 LeNet，是一种经典的卷积神经

网络，它相较常规神经网络具有更小规模的模型参数与更好的特征提取能力，以实现在有限的时间内完成模型的训练，并在图像分类任务上表现优异。对于常规的神经网络，模型的参数量将随着输入数据规格的增长而增长，这显然是不可取的，而卷积神经网络则采用卷积层与池化层进行特征的提取，在减少了参数量的同时增大了训练的可能性。

通过本次实验，我学习了卷积神经网络的基本原理，并学习掌握了MindSpore 这一深度学习框架，从代码角度进行了模型的定义、训练参数的定义、模型的训练、模型的保存与加载、模型的测试，全方面地学习了解了使用神经网络进行特定任务实现的编码流程。同时也通过了一定的参数调整技巧，例如初始参数分布的调整、训练轮数与数据批次的调整等，使模型具有更好的泛化表现。